

AI-DND

CODEX_RULES.md

Rules for Codex and AI Coding Agents

Version 0.1

1. PURPOSE

This document defines strict rules for Codex and any AI coding agent working on AI-DND.

Codex is not the game designer.

Codex is the implementation assistant.

Codex must follow existing documentation and must not invent new game rules without approval.

2. SOURCE OF TRUTH

The following documents are the source of truth:

- PROJECT_VISION.md
- 01_LORE.md
- 02_GAMEPLAY.md
- 03_COMBAT.md
- 04_CHARACTER_SYSTEM.md
- INVENTORY_SYSTEM.md
- MAGIC_SYSTEM.md
- QUEST_SYSTEM.md
- WORLD_MAP_SYSTEM.md
- ECONOMY_SYSTEM.md
- CRAFTING_SYSTEM.md
- FACTION_SYSTEM.md
- UI_UX_SYSTEM.md
- TECHNICAL_ARCHITECTURE.md
- CODEX_RULES.md

If code conflicts with documentation, documentation wins.

3. CODEX ROLE

Codex may:

- create files;
- edit files;
- refactor code;
- fix bugs;
- write tests;
- implement systems described in documentation;
- improve code quality;
- suggest technical improvements.

Codex may not:

- change core design philosophy;
- rewrite lore;
- change combat rules;
- invent new factions;
- simplify mechanics without permission;
- remove systems from the project;
- make AI decide dice results.

4. ABSOLUTE PROJECT RULES

Codex must never violate these rules.

Rule 1

The player may attempt anything.

The interface must not restrict imagination.

Rule 2

The game engine calculates results.

AI only narrates or assists.

Rule 3

AI never decides:

- dice results;
 - hit chance;
 - damage;
 - death;
 - item creation;
 - quest success;
 - world state changes.
-

Rule 4

NPCs know only what they could realistically know.

No NPC is omniscient.

Rule 5

The world continues without the player.

Stone Sleep and Hybrid Time are core systems.

Rule 6

There are no dialogue buttons as the main interaction method.

The player types free text.

Rule 7

Character thoughts are allowed.

Traditional UI hints are not.

Rule 8

The player is not the chosen one.

Power, reputation and knowledge must be earned.

Rule 9

No quest markers.

No floating exclamation marks.

No glowing objective paths.

Rule 10

Consequences are persistent.

The world remembers.

5. DEVELOPMENT ORDER

Codex must follow the development order unless the user explicitly changes it.

Phase 1 — Frontend Prototype

Implement:

1. React + Vite frontend.
2. Main menu.
3. Character creation screen.
4. World map screen.
5. Travel prototype.
6. Event scene screen.
7. Dialogue input.
8. Thought panel.
9. Inventory mockup.
10. Journal mockup.
11. Local mock data.
12. localStorage save.

No backend yet.

No AI yet.

Phase 2 — Core Game Systems

Implement:

1. DiceSystem.
2. IntentSystem mock.
3. RuleValidator.
4. CharacterSystem.
5. InventorySystem.
6. TravelSystem.
7. EventSystem.
8. CombatSystem.
9. InjurySystem.
10. ThoughtSystem.

Phase 3 — Data Driven World

Move content into JSON/data files:

- items;
- skills;
- locations;
- NPCs;
- factions;
- quests;
- events;
- lore.

Phase 4 — Backend

Only after frontend prototype is playable.

Implement:

- Node.js backend;
 - save API;
 - world state API;
 - simulation API;
 - AI bridge.
-

Phase 5 — AI Integration

Only after rules systems exist.

Implement:

- AI NPC dialogue;
 - AI Dungeon Master;
 - AI Narrator;
 - Lore Keeper;
 - memory retrieval;
 - local LLM connection.
-

6. TECH STACK RULES

Use:

- TypeScript;
- React;
- Vite;
- PixiJS where visual game rendering is needed;
- CSS Modules or plain CSS;
- Node.js later;
- PostgreSQL later;
- local LLM later.

Do not use:

- Unity;
 - Unreal;
 - Phaser unless explicitly approved;
 - external paid AI APIs by default;
 - JavaScript files unless unavoidable.
-

7. FILE STRUCTURE RULES

Codex must respect this structure:

```
AI-DND/  
├─ client/  
├─ server/  
└─ shared/
```

```
|— data/
|— docs/
|— assets/
|— tools/
└─ README.md
```

Client structure:

```
client/
|— public/
|— src/
|   |— app/
|   |— assets/
|   |— components/
|   |— data/
|   |— engine/
|   |— features/
|   |— game/
|   |— scenes/
|   |— state/
|   |— styles/
|   |— systems/
|   |— types/
|   └─ main.tsx
```

Codex must not create random folders.

If a new folder is needed, explain why before creating it.

8. CODE STYLE RULES

Use strict TypeScript.

Prefer small files.

Recommended maximum:

300 lines per file.

If a file grows too large, split it.

Use:

- PascalCase for React components;
 - PascalCase for types and interfaces;
 - camelCase for functions and variables;
 - kebab-case for asset filenames.
-

9. ARCHITECTURE RULES

UI components must not contain core game logic.

Game systems must be separate from UI.

Bad:

React component calculates combat damage.

Good:

React component calls CombatSystem.

10. SYSTEM RESULT RULES

Systems must return structured results.

Bad:

```
return "You hit the enemy.";
```

Good:

```
return {
  success: true,
  type: "combat.attack",
  target: "torso",
  damage: 8,
  effects: ["bleeding_light"]}
};
```

Narration happens separately.

11. AI RULES

AI modules may:

- generate dialogue;
- generate narration;
- summarize events;
- suggest possible interpretations;
- help parse intent.

AI modules may not:

- decide real outcomes;
- modify world state directly;
- create items directly;
- change stats directly;
- override dice;
- ignore rules.

12. INTENT SYSTEM RULES

Player input must be interpreted as intention, not command.

Example:

Player writes:

"I try to strike his head."

System must parse:

```
{
  "type": "combat.attack",
  "target": "head",
  "attempted": true
}
```

Then RuleValidator determines if it is possible.

13. RULE VALIDATOR RULES

RuleValidator checks:

- equipment;
- skill;
- training;
- injuries;
- fatigue;
- context;
- target availability;
- physical possibility.

If action is impossible, it must return a structured failure.

Example:

```
{
  allowed: false,
  reason: "no_sword_equipped",
  fallbackAction: "reach_for_missing_weapon"
}
```

The UI must not show raw technical error text to the player.

14. DICE SYSTEM RULES

DiceSystem must be independent and testable.

DiceSystem handles:

- d20;
- damage dice;
- critical success;
- critical failure;
- modifiers.

AI must never generate dice results.

15. COMBAT SYSTEM RULES

Combat must support:

- initiative;
- body part targeting;
- training-based target access;
- damage types;
- armor;
- injuries;
- bleeding;
- fractures;
- surrender;
- escape.

At level one / beginner training, the player can only reliably attack torso.

Advanced targeting requires training.

16. THOUGHT SYSTEM RULES

Thoughts are immersive hints.

They must look like character thoughts, not UI commands.

Good:

- He seems nervous.
- My arm hurts.
- This road feels dangerous.

Bad:

Click here to intimidate.
Press 1 to attack.
Available action: negotiate.

17. WORLD MAP RULES

The player travels on a global map.

The player does not freely control a character in real-time locations during the main prototype.

Travel creates events.

Events transition into scenes.

18. EVENT SCENE RULES

Event scenes contain:

- background/environment;
 - NPC or enemy;
 - narration;
 - thoughts;
 - free text input;
 - possible transition to combat, dialogue, escape, investigation.
-

19. SAVE RULES

The save represents the world, not just the player.

Save data must eventually include:

- player;
- NPCs;
- memories;
- rumors;
- history;
- world events;
- factions;
- economy;
- Stone Sleep state.

During frontend prototype, localStorage is allowed.

20. STONE SLEEP RULES

When the player is offline:

- the character enters Stone Sleep;
- world simulation continues;
- time is compressed;
- player returns to changed world.

Stone Sleep must be treated as lore, not only technical offline progression.

21. LORE RULES

Codex must not introduce:

- modern technology;
- cars;
- internet;
- phones;
- robots;
- science fiction explanations;
- references that break dark fantasy tone.

Unless explicitly approved.

22. NPC KNOWLEDGE RULES

NPC responses must respect:

- personal knowledge;
- local knowledge;
- profession;
- rumors;
- secrets;
- memory.

A farmer should not explain ancient forbidden magic unless the lore gives a reason.

23. UI RULES

The UI must use:

- dark fantasy style;
- parchment;
- wood;
- iron;
- candlelight;
- readable text.

Avoid:

- modern flat UI;
 - neon buttons;
 - mobile-game style;
 - bright arcade colors.
-

24. QUEST RULES

Quests are problems, not checklist missions.

Quest journal records clues and memories.

Bad:

Kill 5 wolves.

Good:

The hunter believes wolves are attacking travelers near the old forest road.

25. INVENTORY RULES

Inventory is physical.

Items have:

- weight;

- condition;
- ownership;
- quality;
- history.

Do not create infinite inventory unless explicitly approved.

26. ECONOMY RULES

Prices are dynamic.

Economy responds to:

- supply;
 - demand;
 - war;
 - famine;
 - disease;
 - trade routes;
 - reputation.
-

27. ERROR HANDLING RULES

The app should not crash on invalid player input.

Invalid input should become:

- failed attempt;
 - misunderstood action;
 - partial action;
 - natural narration.
-

28. TESTING RULES

Codex should create tests for pure systems.

Priority tests:

- DiceSystem;
- RuleValidator;
- IntentSystem;

- CombatSystem;
 - InventorySystem;
 - TravelSystem;
 - ThoughtSystem.
-

29. DOCUMENTATION RULES

When Codex creates a major system, it must:

- follow existing docs;
 - update relevant README or comments if needed;
 - not rewrite design documents without instruction.
-

30. FORBIDDEN SHORTCUTS

Codex must not:

- replace free text input with buttons;
 - hardcode all gameplay into one component;
 - create one giant App.tsx;
 - make AI responsible for rules;
 - skip RuleValidator;
 - skip DiceSystem;
 - ignore body part system;
 - ignore Stone Sleep;
 - ignore world persistence.
-

31. WHEN UNCERTAIN

If Codex is uncertain, it should:

1. Follow documentation.
 2. Choose the simplest implementation that preserves architecture.
 3. Avoid inventing new lore or mechanics.
 4. Leave TODO comments only when unavoidable.
 5. Ask for clarification only if implementation would otherwise violate core rules.
-

32. CODEX TASK FORMAT

Every task given to Codex should include:

- goal;
- files to create or modify;
- constraints;
- acceptance criteria.

Example:

```
Goal:
Create MainMenu screen.

Files:
client/src/scenes/MainMenuScene.tsx
client/src/components/FantasyButton.tsx
client/src/styles/main-menu.css

Constraints:
Use existing assets.
Do not add backend.
Do not add AI.
Do not change architecture.

Acceptance:
The app opens to main menu.
Buttons are visible.
New Game navigates to character creation.
```

33. FINAL RULE

Codex implements.

Documentation defines.

The user approves.

No AI agent owns the design of AI-DND.